

DJANGO REST API CRUD OPERATION PROJECT

Project Name : `restapiproject`

App Name : `apiapp`

WHAT IS REST API?

REST API is used for communication between applications.

HTTP Methods:

- GET → Fetch Data
 - POST → Insert Data
 - PUT → Update Data
 - DELETE → Delete Data
-

STEP 1 : OPEN PROJECT IN VS CODE

Open CMD:

```
cd restapiproject  
  
code .
```

Activate Virtual Environment:

```
venv\Scripts\activate
```

Run Server:

```
python manage.py runserver
```

STEP 2 : INSTALL DJANGO REST FRAMEWORK

```
pip install djangorestframework
```

STEP 3 : ADD APP IN settings.py

File:

```
restapiproject/settings.py
```

Add:

```
INSTALLED_APPS = [  
    ...  
    'rest_framework',  
    'apiapp',  
]
```

STEP 4 : CREATE MODEL

File:

```
apiapp/models.py
```

Code:

```
from django.db import models  
  
class Student(models.Model):  
  
    name = models.CharField(max_length=100)  
    email = models.EmailField()  
    mobile = models.CharField(max_length=15)  
    city = models.CharField(max_length=100)  
  
    def __str__(self):  
        return self.name
```

STEP 5 : MAKE MIGRATIONS

```
python manage.py makemigrations
```

```
python manage.py migrate
```

STEP 6 : CREATE SERIALIZER

WHAT IS SERIALIZER?

Serializer converts:

- Python/Django model data → JSON
- JSON data → Python objects

Create File:

```
apiapp/serializers.py
```

Code:

```
from rest_framework import serializers
from .models import Student

class StudSerial(serializers.ModelSerializer):

    class Meta:

        model = Student
        fields = '__all__'
```

STEP 7 : CREATE VIEWS

File:

```
apiapp/views.py
```

Code:

```
from django.shortcuts import render
from rest_framework.decorators import api_view
from rest_framework.response import Response
from rest_framework import status
from .models import Student
from .serializers import StudSerial
import requests
```

```

# =====
# GET ALL DATA
# =====

@api_view(['GET'])
def getdata(request):

    student = Student.objects.all()

    serial = StudSerial(student, many=True)

    return Response(serial.data)


# =====
# SEARCH SINGLE DATA
# =====

@api_view(['GET'])
def searchdata(request, pk):

    try:

        student = Student.objects.get(id=pk)

    except Student.DoesNotExist:

        return Response(status=status.HTTP_404_NOT_FOUND)

    serial = StudSerial(student)

    return Response(serial.data)


# =====
# SAVE DATA
# =====

@api_view(['POST'])
def savedata(request):

    serial = StudSerial(data=request.data)

    if serial.is_valid():

        serial.save()

    return Response(
        {
            "message": "Data Inserted Successfully"

```

```

        },
        status=status.HTTP_201_CREATED
    )

else:

    return Response(
        serial.errors,
        status=status.HTTP_400_BAD_REQUEST
    )

# =====
# UPDATE DATA
# =====

@api_view(['PUT', 'GET'])
def updatedata(request, pk):

    try:

        student = Student.objects.get(id=pk)

    except Student.DoesNotExist:

        return Response(status=status.HTTP_404_NOT_FOUND)

    serial = StudSerial(student, data=request.data)

    if serial.is_valid():

        serial.save()

        return Response(
            {
                "message": "Data Updated Successfully"
            }
        )

    return Response(serial.errors)

# =====
# DELETE DATA
# =====

@api_view(['DELETE', 'GET'])
def deletedata(request, pk):

    try:

```

```

        student = Student.objects.get(id=pk)

    except Student.DoesNotExist:

        return Response(status=status.HTTP_404_NOT_FOUND)

    student.delete()

    return Response(
        {
            "message": "Data Deleted Successfully"
        }
    )

# =====
# INDEX PAGE
# =====

def index(request):

    url = "http://127.0.0.1:8000/getdata/"

    req = requests.get(url)

    data = req.json()

    context = {
        'data': data
    }

    return render(request, 'index.html', context)

```

STEP 8 : CREATE URLS

File:

```
apiapp/urls.py
```

Code:

```

from django.urls import path
from . import views

urlpatterns = [

```

```
path('', views.index),

path('getdata/', views.getdata),

path('searchdata/<int:pk>', views.searchdata),

path('savedata/', views.savedata),

path('updatedata/<int:pk>', views.updatedata),

path('deletedata/<int:pk>', views.deletedata),
]
```

STEP 9 : MAIN URLS

File:

```
restapiproject/urls.py
```

Code:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [

    path('admin/', admin.site.urls),

    path('', include('apiapp.urls')),
]
```

STEP 10 : INSTALL REQUESTS LIBRARY

```
pip install requests
```

STEP 11 : CREATE TEMPLATE FOLDER

Inside `apiapp` create folder:

templates

Inside templates create file:

index.html

STEP 12 : DESIGN index.html

File:

apiapp/templates/index.html

Code:

```
<!DOCTYPE html>
<html>

<head>

    <title>Student Data</title>

    <style>

        body{
            font-family:Arial;
            background:#f4f4f4;
            padding:40px;
        }

        h1{
            text-align:center;
            color:#333;
        }

        table{
            width:100%;
            border-collapse:collapse;
            background:white;
        }

        th,td{
            border:1px solid #ccc;
            padding:12px;
            text-align:center;
```



```

    }

    th{
        background:#007bff;
        color:white;
    }

    tr:nth-child(even){
        background:#f2f2f2;
    }

</style>

</head>

<body>

    <h1>Student Data Table</h1>

    <table>

        <tr>

            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
            <th>Mobile No</th>
            <th>City</th>

        </tr>

        {% for i in data %}

            <tr>

                <td>{{i.id}}</td>
                <td>{{i.name}}</td>
                <td>{{i.email}}</td>
                <td>{{i.mobile}}</td>
                <td>{{i.city}}</td>

            </tr>

        {% endfor %}

    </table>

</body>

</html>

```

STEP 13 : RUN SERVER

```
python manage.py runserver
```

STEP 14 : TEST API URLs

1 GET DATA

URL:

```
http://127.0.0.1:8000/getdata/
```

Method:

```
GET
```

Purpose:

Fetch all records.

2 SEARCH DATA

URL:

```
http://127.0.0.1:8000/searchdata/1/
```

Method:

```
GET
```

Purpose:

Fetch specific data using ID.

3 SAVE DATA

URL:

```
http://127.0.0.1:8000/savedata/
```

Method:

```
POST
```

JSON Data:

```
{
  "name": "Richa",
  "email": "richa@gmail.com",
  "mobile": "9876543210",
  "city": "Ahmedabad"
}
```

Purpose:

Insert new data.

4 UPDATE DATA

URL:

```
http://127.0.0.1:8000/updatedata/1/
```

Method:

```
PUT
```

JSON Data:

```
{
  "name": "Richa Parmar",
  "email": "richa@gmail.com",
  "mobile": "9999999999",
}
```

```
{  "city": "Surat"}
```

Purpose:

Update existing data.

5 DELETE DATA

URL:

```
http://127.0.0.1:8000/deletedata/1/
```

Method:

```
DELETE
```

Purpose:

Delete specific data using ID.

STEP 15 : API DOCUMENTATION FORMAT

Open Excel and create table:

Sr No	URL	Method	Parameter	Remarks
1	/getdata/	GET	No	Fetch all data
2	/searchdata/1/	GET	ID	Fetch single data
3	/savedata/	POST	JSON Body	Insert new data
4	/updatedata/1/	PUT	ID + JSON	Update existing data
5	/deletedata/1/	DELETE	ID	Delete data

CRUD OPERATION

Operation	Meaning
Create	Insert Data

Operation	Meaning
Read	Fetch Data
Update	Modify Existing Data
Delete	Remove Data

INTERVIEW QUESTIONS AND ANSWERS

1 What is REST API?

REST API is a communication method between frontend and backend using HTTP methods.

2 What is Serializer?

Serializer converts Django/Python objects into JSON format and JSON into Python objects.

3 Difference Between GET and POST

GET	POST
Fetch Data	Insert Data
Data visible in URL	Data sent in Body
Less Secure	More Secure

4 What is JSON?

JSON means JavaScript Object Notation.

Used for storing and transferring data in APIs.

Example:

```
{
  "name": "Richa"
}
```

5 Why Use REST API?

- Frontend and Backend Communication
 - Mobile App Integration
 - Third Party Integration
 - Data Exchange in JSON Format
-

PROJECT FLOW

```
User Request
  ↓
URL
  ↓
View Function
  ↓
Serializer
  ↓
Database
  ↓
JSON Response
```

IMPORTANT COMMANDS

Create App

```
python manage.py startapp apiapp
```

Run Server

```
python manage.py runserver
```

Migration Commands

```
python manage.py makemigrations
```

```
python manage.py migrate
```

FINAL OUTPUT

This project includes:

- ✓ Django REST API
- ✓ CRUD Operations
- ✓ Serializer
- ✓ JSON Response
- ✓ API Testing
- ✓ HTML Frontend
- ✓ API Documentation
- ✓ Interview Preparation Project